File	Purpose
test/fixtures/comments.yml	Sample comments for use in testing

First, take a look at `app/models/comment.rb`:

```
class Comment < ActiveRecord::Base
  belongs_to :article
end
```

This is very similar to the `Article` model that you saw earlier. The difference is the line `belongs_to :article`, which sets up an Active Record *association*. You'll learn a little about associations in the next section of this guide.

In addition to the model, Rails has also made a migration to create the corresponding database table:

```
class CreateComments < ActiveRecord::Migration
  def change
    create_table :comments do |t|
      t.string :commenter
      t.text :body

      # this line adds an integer column called `article_id`.
      t.references :article, index: true

      t.timestamps null: false
    end
    add_foreign_key :comments, :articles
  end
end
```

The `t.references` line sets up a foreign key column for the association between the two models. An index for this association is also created on this column. Go ahead and run the migration:

```
$ bin/rake db:migrate
```

Rails is smart enough to only execute the migrations that have not already been run against the current database, so in this case you will just see:

```
==  CreateComments: migrating
=============================================
-- create_table(:comments)
   -> 0.0115s
-- add_foreign_key(:comments, :articles)
   -> 0.0000s
==  CreateComments: migrated (0.0119s)
========================================
```

6.2 Associating Models

Active Record associations let you easily declare the relationship between two models. In the case of comments and articles, you could write out the relationships this way:

- Each comment belongs to one article.
- One article can have many comments.

In fact, this is very close to the syntax that Rails uses to declare this association. You've already seen the line of code inside the `Comment` model (app/models/comment.rb) that makes each comment belong to an Article:

```
class Comment < ActiveRecord::Base
  belongs_to :article
end
```

You'll need to edit `app/models/article.rb` to add the other side of the association: